

```
> kind get clusters
kind
```

You can also get it's *kubeconfig* via the following command:

```
> kind get kubeconfig | head
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FUR
S0tLS0tCk1JSUM1ekNDQWMrZ0F3 SUJBZ0lCQURBTKJna3Foa2l
HOXcwQkFRc0ZBREFTVjNd0VRWURWUVERXdwcm
RXSmwKY201bGRHVnpNQjRYRF RJeU1ESXd0REUxTlRNeU1sb1hEVE15TURJd0
1qRTFOV<Snipped>
OG9QRDNoYnRraFJ4MQotLS0tL UVORCBDRVJUSUZJQ0FURS0tLS0tCg==
    server: https://127.0.0.1:38471
    name: kind-kind
contexts:
- context:
    cluster: kind-kind
    user: kind-kind
```

You can use the `--wait` flag if you want to wait till the control plane is in a reachable state. Since Kind is mostly used for testing, you cannot actually update it. Let's say we want to have a declarative setup with this configuration for our Kind cluster:

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
  kubeadmConfigPatches:
  - |
    kind: InitConfiguration
    nodeRegistration:
      kubeletExtraArgs:
        node-labels: "ingress-ready=true"
  extraPortMappings:
  - containerPort: 80
    hostPort: 80
    protocol: TCP
  - containerPort: 443
    hostPort: 443
    protocol: TCP
```

The above configuration creates a Kind cluster with `extraPortMappings` and `node-labels`. The port mappings are required to allow localhost to make requests to the Ingress controller. Let's try to apply the above manifest (but note that it's going to fail):

```
> kind create cluster --config ./cluster.yaml
```

```
ERROR: failed to create cluster: node(s) already exist for
a cluster with the name "kind"
```

So we have to delete and create a new one. Let's delete it first:

```
❏ kind delete cluster
Deleting cluster "kind" ...
```

Let's create the cluster using our declarative setup now:

```
> kind create cluster --config ./cluster.yaml
Creating cluster "kind" ...
> Ensuring node image (kindest/node:v1.21.1) >
> Preparing nodes >
> Writing configuration >
> Starting control-plane >
> Installing CNI >
> Installing StorageClass >
Set kubectl context to "kind-kind"
```

You can now use your cluster with:

```
kubectl cluster-info --context kind-kind
```

You can also specify your k8s version in the above manifest by specifying the proper Docker image, which you can find in its GitHub release page:

```
- role: control-plane
  image: kindest/node:v1.21.1@sha256:69860bda5563ac81e3c005
7d654b5253219618a22ec3a346306239bba8cfa1a6
```

Cluster interaction

We need to interact with the cluster to get the actual job done. This could be deploying your application or doing maintenance operations on your node. You can use the usual *kubectl* to interact with your cluster:

```
> kubectl get nodes
NAME                STATUS    ROLES          AGE      VERSION
kind-control-plane  Ready    control-plane,master 95s
v1.21.1
```

Now let's check the various name spaces present:

```
> kubectl get namespaces -A
NAME                STATUS    AGE
default             Active    3m1s
kube-node-lease     Active    3m2s
kube-public         Active    3m2s
kube-system         Active    3m2s
```